

Object Oriented Programming

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Email: usman@uetmardan.edu.pk

COURSE LEARNING OUTCOMES

Course Learning Outcome	CLO Statements	Domain	Taxonomy level	PLOs
CLO-1	Understand principles of object-oriented paradigm	C	2	2
CLO-2	Identify the objects & their relationships to build object-oriented solution	C	3	3
CLO-3	Model a solution for a given problem using object-oriented principles	C	3	4
CLO-4	Evaluate an object-oriented solution	C	4	3

SOME RECOMMENDED TEXTBOOKS

- **C++ How to Program, By: Harvey Deitel & Paul Deitel, Edition: 10th**
- **Object Oriented Programming in C++ By: Robert Lafore, Edition: 4th ,
Publisher: SAMS Publishing, ISBN: 0672323087, Year of Publication: 2001.**
- **Java How to Program, By: Harvey Deitel & Paul Deitel, Edition: 9th,
Publisher: Prentice Hall, ISBN: 0132575663, Year of Publication: 2011.**
- **Starting Out with C++ from Control Structures to Objects, 9th Edition,
Tony Gaddis**
- **Let Us C++ by Yashwant kanatkar**

TENTATIVE GRADING CRITERIA

- Final Term 50%
- Mid Term 25%
- Sessional Marks 25%
 - Quizzes 08%
 - Assignments 10%
 - Project 07%



COURSE OUTLINE

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING CONCEPTS

Constructors, Destructors

Member functions

Data Encapsulation

Function overloading, operator overloading

Abstraction

Dynamic memory management

Inheritance, polymorphism

Exception handling

Standard template library, object streams

LET'S START



Computer Languages

Machine language

- Only language computer directly understands
- Defined by hardware design
 - Machine-dependent
- Generally consist of strings of numbers
 - Ultimately 0s and 1s
- Instruct computers to perform elementary operations
 - One at a time
- Cumbersome for humans
- Example:

```
11000101011
00101101011
11111000010
```

Computer Languages

Assembly language

- English-like abbreviations representing elementary computer operations
- Clearer to humans
- Incomprehensible to computers
 - Translator programs (assemblers)
 - Convert to machine language
- Example:

```
LOAD  BASEPAY
ADD   OVERPAY
STORE GROSSPAY
```

Computer Languages

High-level languages

- Similar to everyday English, use common mathematical notations
- Single statements accomplish substantial tasks
 - Assembly language requires many instructions to accomplish simple tasks
- Uses translator programs (compilers)
 - Convert to machine language
- Interpreter programs
 - Directly execute high-level language programs
- Example:

grossPay = basePay + overTimePay

Programming Approach

Structured programming (1960s)

- Disciplined approach to writing programs
- Clear, easy to test and debug, and easy to modify
- Extensively uses of the structured control flow consist of selection (if/then/else) and repetition (while and for) block structures, and subroutines.

Pascal

- 1971: Niklaus Wirth

Ada

- 1970s - early 1980s: US Department of Defense (DoD)
- Multitasking
 - Programmer can specify many activities to run in parallel

Programming Approach

Objects

- Objects are reusable software components that model real world items
- These are meaningful software units e.g.
 - Date objects, time objects, paycheck objects, invoice objects, audio objects, video objects, file objects, record objects, etc.
 - Any noun can be represented as an object
- Objects are more understandable, better organized and easier to maintain than procedural programming

WHAT IS OBJECT ORIENTED PROGRAMMING?

- OOP is a programming paradigm or programming style centered around objects rather than functions.
- This style of programming is not new but is followed since 1970s.
- Object Oriented Programming is not a programming language or a tool, but it is a framework, a style, a way of doing programming
- There are many languages which support OOP framework

LANGUAGES

C++

C#

JAVA

Ruby

Python

JavaScript

And many more

HISTORY OF C AND C++

History of C

- Evolved from two other programming languages
 - BCPL and B
 - “Typeless” languages
- Dennis Ritchie (Bell Laboratories)
 - Added data typing, other features
- Development language of UNIX
- Hardware independent
 - Portable programs
- 1989: ANSI standard
- 1990: ANSI and ISO standard published
 - ANSI/ISO 9899: 1990

HISTORY OF C AND C++

History of C++

- Extension of C
- Early 1980s: Bjarne Stroustrup (Bell Laboratories)
- Provides capabilities for object-oriented programming
 - Objects: reusable software components
 - Model items in real world
 - Object-oriented programs
 - Easy to understand, correct and modify
- Hybrid language
 - C-like style
 - Object-oriented style
 - Both

C++ STANDARD LIBRARY

C++ programs

- Built from pieces called classes and functions

C++ standard library

- Rich collections of existing classes and functions

“Building block approach” to creating programs

- “Software reuse”

JAVA

Java

- 1991: Sun Microsystems
 - Green project
- 1995: Sun Microsystems
 - Formally announced Java at trade show
- Web pages with dynamic and interactive content
- Develop large-scale enterprise applications
- Enhance functionality of web servers
- Provide applications for consumer devices
 - Cell phones, pagers, personal digital assistants, ...

KEY OOP-RELATED FEATURES LACKING IN JAVA COMPARED TO C++

- Multiple Inheritance of Classes
- Operator Overloading
- Pointers and Pointer Arithmetic
- Structures and Unions
- Templates

What is OOP?

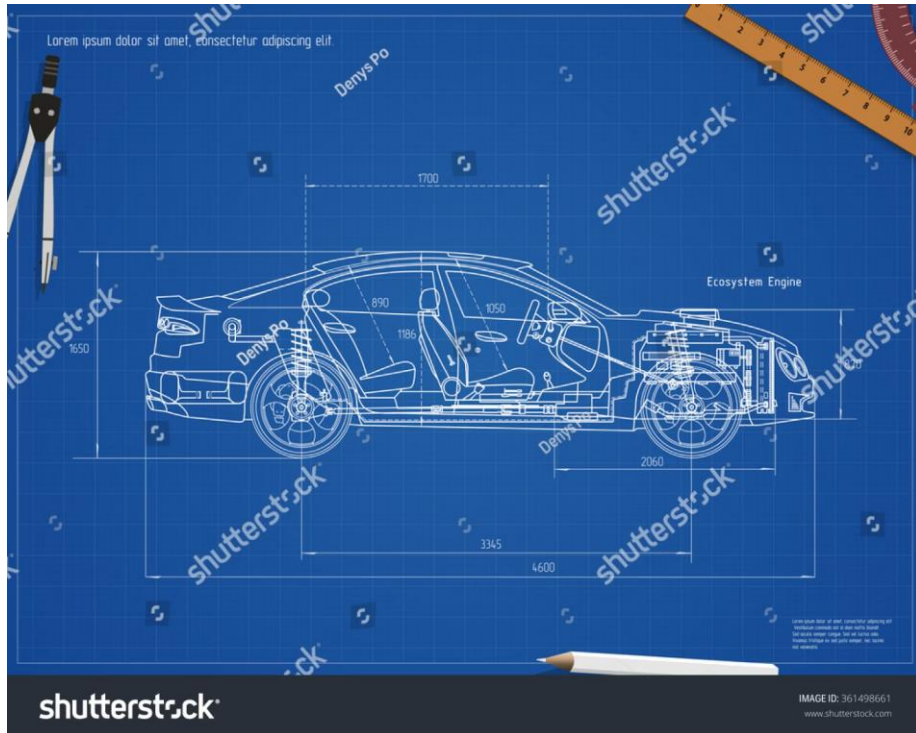
A paradigm shift in software design

- Object-Oriented Programming (OOP) represents a fundamental change in how we approach software development.
- Instead of focusing on actions and logic, OOP emphasizes data and the objects that contain it, leading to more intuitive and organized code.



Classes and Objects

The building blocks of OOP



Classes: The Blueprints

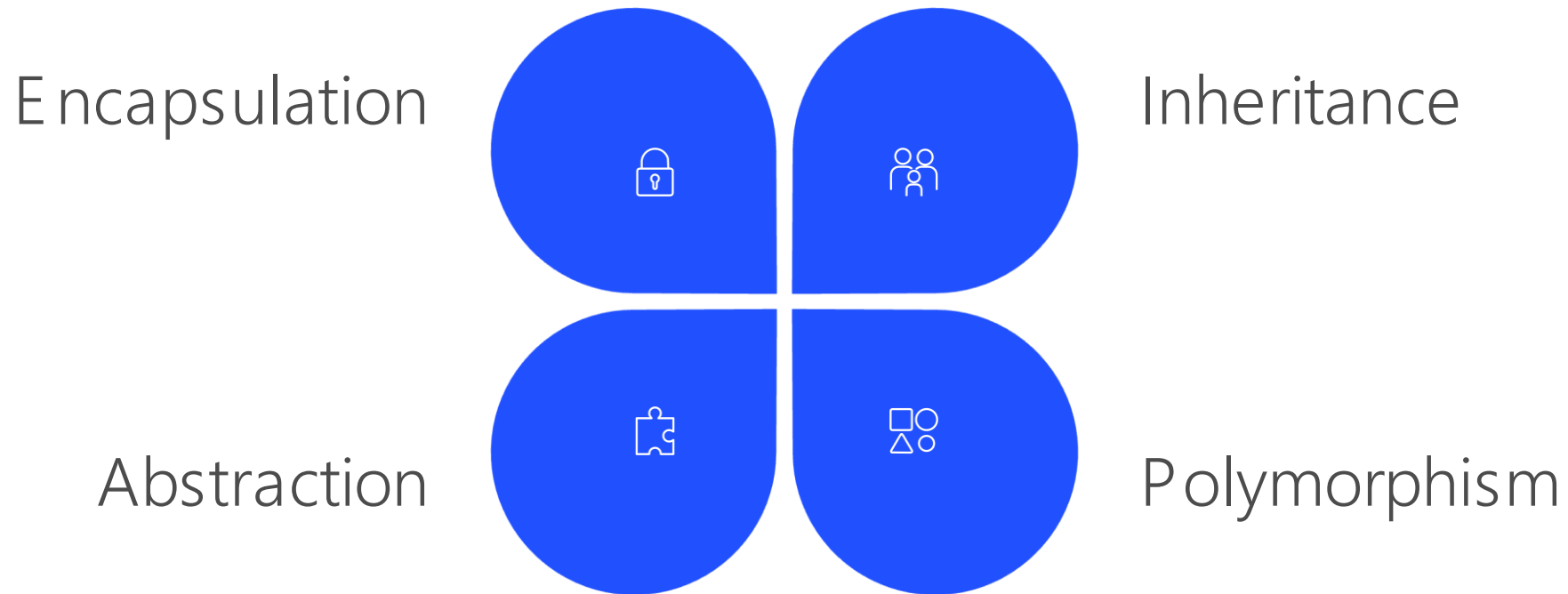
A **class** is a template or blueprint for creating objects. It defines the properties (attributes) and behaviors (methods) that an object of that class will have. Think of it as the design for a car.



Objects: The Instances

An **object** is an instance of a class. It's a real-world entity created from the blueprint. Each object has its own unique set of attribute values. The car itself, built from the blueprint, is an object.

The Four Pillars of OOP



These four core principles form the foundation of Object-Oriented Programming, guiding developers to create robust, modular, and maintainable software systems.

Basics of a Typical C++ Environment

C++ systems

- Program-development environment
- Language
- C++ Standard Library
 - A collection of classes and functions, which are written in the core language and part of the C++ ISO Standard itself

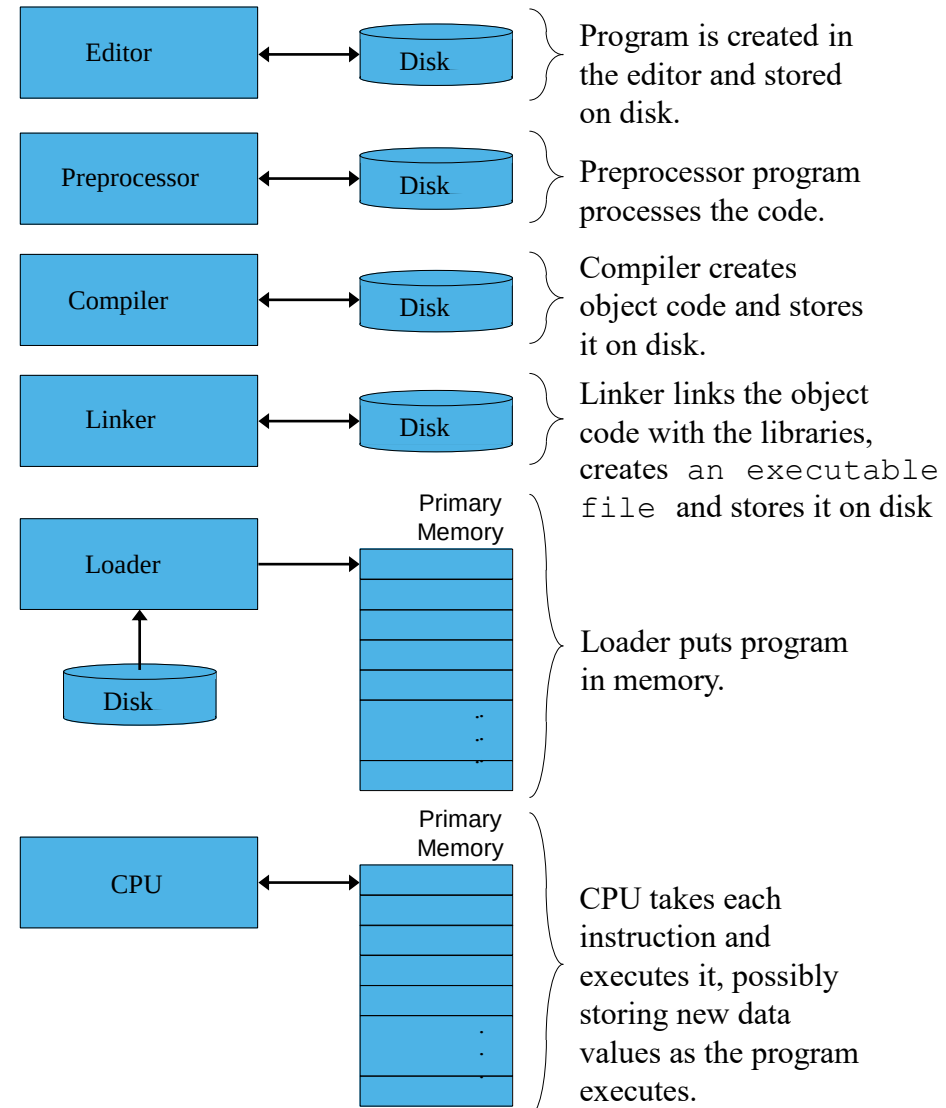
C++ program names extensions

- .cpp
- .cxx
- .cc
- .C

Basics of a Typical C++ Environment

Phases of C++ Programs:

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



Basics of a Typical C++ Environment

Common Input/output functions

`cin`

- Standard input stream
- Normally keyboard

`cout`

- Standard output stream
- Normally computer screen

`cerr`

- Standard error stream
- Display error messages

A Simple Program: Printing a Line of Text

Before writing the programs

Comments

- Document programs
- Improve program readability
- Ignored by compiler
- Single-line comment
 - Use C's comment `/* .. */` OR Begin with `//`

Preprocessor directives

- Processed by preprocessor before compiling
- Begin with `#`

1 // A first program in C++.

3 #include <iostream>

4

5 // function main returns an integer value.

6 int main()

7 {

8

std::cout << "Welcome to C++!\n";

9

10 return 0; // indicates program terminated successfully

11

12 } // end function main

Single line comments
Preprocessor directive to include input/output stream header file <iostream>.

Function **main** returns an integer value.

Left brace { begins function body.
program..

Statements end with a semicolon ;.

Stream insertion operator.
Name **cout** belongs to namespace **std**.

Corresponding right brace } ends function body.

Keyword **return** is one of several means to exit function; value **0** indicates program terminated successfully.

Welcome to C++!

A Simple Program: Printing a Line of Text

Standard output stream object

- `std::cout`
- “Connected” to screen
- `<<`
 - Stream insertion operator
 - Value to right (right operand) inserted into output stream

Namespace

- `std::` specifies using name that belongs to “namespace” `std`
- `std::` removed through use of `using` statements

Escape characters

- `\`
- Indicates “special” character output

Basic Structure of a C++ Program

1. Preprocessor Directives

- `#include <iostream>` // For input/output (cin, cout)
- `#include <string>` // For string operations
- `#include <cmath>` // For mathematical functions
- `#include <vector>` // For vectors (dynamic arrays)
- Start with `#include`
- Import libraries/header files needed for your program
- `< >` for standard libraries
- `" "` for custom header files: `#include "myheader.h"`

Common Headers:

- `<iostream>` - Input/Output (cin, cout)
- `<string>` - String handling
- `<cmath>` - Math functions (sqrt, pow, etc.)
- `<vector>` - Dynamic arrays
- `<fstream>` - File operations
- `<iomanip>` - Formatting output

2. Namespace Declaration

`using namespace std;`

- Allows you to use `cout` instead of `std::cout`
- Optional but commonly used
- Without it, you'd write: `std::cout << "Hello";`

3. Global Variables and Constants (Optional)

`const double PI = 3.14159;` *// Constant (can't be changed)*

`int counter = 0;` *// Global variable (avoid if possible)*

- Declared outside all functions
- Accessible from anywhere in the program
- Constants use `const` keyword

4. Function Prototypes (Optional)

`int add(int a, int b);` *// Returns int, takes 2 ints*

`void printMessage();` *// Returns nothing*

`double calculateArea(double r);` *// Returns double, takes double*

- Tells compiler about functions before they're defined
- Needed if function definition comes after `main()`
- Format: `return_type function_name(parameters);`

5. Main Function (Required)

```
int main() { // Your code here  
return 0; // 0 means successful execution }
```

- **Entry point** of every C++ program
- Must exist in every program
- int means it returns an integer
- return 0; indicates successful execution
- Program execution starts here

6. Function Definitions

```
int add(int a, int b) {  
    int result = a + b;  
    return result;  
}
```

- Actual implementation of functions
- Can be before or after main() (use prototypes if after)

A Simple Program: Printing a Line of Text

Escape Sequence	Description
<code>\n</code>	Newline. Position the screen cursor to the beginning of the next line.
<code>\t</code>	Horizontal tab. Move the screen cursor to the next tab stop.
<code>\r</code>	Carriage return. Position the screen cursor to the beginning of the current line; do not advance to the next line.
<code>\a</code>	Alert. Sound the system bell.
<code>\\</code>	Backslash. Used to print a backslash character.
<code>\"</code>	Double quote. Used to print a double quote character.

Basics of a C++ Program

Variables

- Named location in memory where value can be stored
- Common data types
 - `int` - integer numbers
 - `char` - characters
 - `double` - floating point numbers
- Declare variables with name and data type before use

```
int integer1;  
int integer2;  
int sum;
```
- Can declare several variables of same type in one declaration
 - Comma-separated list

```
int integer1, integer2, sum;
```

Basics of a C++ Program

Variable names

- Correspond to actual locations in computer's memory
- Every variable has name, type, size and value
- When new value placed into variable, overwrites previous value

```
std::cin >> integer1;
```

Assume user entered 45

integer1	45
----------	----

```
std::cin >> integer2;
```

Assume user entered 72

integer1	45
integer2	72

```
sum = integer1 + integer2;
```

integer1	45
integer2	72
sum	117

Basics of a C++ Program

Input stream object

- **>> (stream extraction operator)**
 - Used with `std::cin`
 - Waits for user to input value, then press *Enter* (Return) key
 - Stores value in variable to right of operator
 - Converts value to variable data type

= (assignment operator)

- Assigns value to variable
- Binary operator (two operands)
- Example:

```
sum = variable1 + variable2;
```

```
3  #include <iostream>
4
5  // function main begins program execution
6  int main()
7  {
```

Declare integer variables.

```
8      int integer1; // first number to be input by user
9      int integer2; // second number to be input by user
10     int sum; // variable in which sum will be stored
```

Use stream extraction operator with standard input stream to obtain user input.

```
11
12     std::cout << "Enter first integer\n"; // prompt
13     std::cin >> integer1; // read an integer
```

```
14
15     std::cout << "Enter second integer\n"; // prompt
16     std::cin >> integer2; // read an integer
```

Calculations can be performed in output statements: alternative for lines 18 and 20:

```
17
18     sum = integer1 + integer2; // assign sum
```

```
std::cout << "Sum is " << integer1 + integer2 << std::endl;
```

```
19
20     std::cout << "Sum is " << sum << std::endl; // print sum
```

```
21
22     return 0; // indicate that program ended successfully
```

```
23
24 }
```

Concatenating, chaining or cascading stream insertion operations.

Basics of a C++ Program

Arithmetic calculations

- ***** : Multiplication
- **/** : Division
 - Integer division truncates(shorten) remainder
 - **7 / 5** evaluates to 1
- **%** : Modulus operator returns remainder
 - **7 % 5** evaluates to 2

Operator(s)	Operation(s)	Order of evaluation (precedence)
()	Parentheses	Evaluated first. If the parentheses are nested, the expression in the innermost pair is evaluated first. If there are several pairs of parentheses “on the same level” (i.e., not nested), they are evaluated left to right.
* , / , or %	Multiplication Division Modulus	Evaluated second. If there are several, they re evaluated left to right.
+ or -	Addition Subtraction	Evaluated last. If there are several, they are evaluated left to right.

Basics of a C++ Program

Standard algebraic equality operator or relational operator	C++ equality or relational operator	Example of C++ condition	Meaning of C++ condition
<i>Relational operators</i>			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
<i>Equality operators</i>			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Basics of a C++ Program

C++ keywords

Cannot be used as identifiers or variable names

C++ Keywords

Keywords common to the C and C++ programming languages

<code>auto</code>	<code>break</code>	<code>case</code>	<code>char</code>	<code>const</code>
<code>continue</code>	<code>default</code>	<code>do</code>	<code>double</code>	<code>else</code>
<code>enum</code>	<code>extern</code>	<code>float</code>	<code>for</code>	<code>goto</code>
<code>if</code>	<code>int</code>	<code>long</code>	<code>register</code>	<code>return</code>
<code>short</code>	<code>signed</code>	<code>sizeof</code>	<code>static</code>	<code>struct</code>
<code>switch</code>	<code>typedef</code>	<code>union</code>	<code>unsigned</code>	<code>void</code>
<code>volatile</code>	<code>while</code>			

C++ only keywords

<code>asm</code>	<code>bool</code>	<code>catch</code>	<code>class</code>	<code>const_cast</code>
<code>delete</code>	<code>dynamic_cast</code>	<code>explicit</code>	<code>false</code>	<code>friend</code>
<code>inline</code>	<code>mutable</code>	<code>namespace</code>	<code>new</code>	<code>operator</code>
<code>private</code>	<code>protected</code>	<code>public</code>	<code>reinterpret_cast</code>	
<code>static_cast</code>	<code>template</code>	<code>this</code>	<code>throw</code>	<code>true</code>
<code>try</code>	<code>typeid</code>	<code>typename</code>	<code>using</code>	<code>virtual</code>
<code>wchar_t</code>				

Basics of a C++ Program

if structure

- Make decision based on truth or falsity of condition
 - If condition met, body executed
 - Else, body not executed

Equality and relational operators

- Equality operators
 - Same level of precedence
- Relational operators
 - Same level of precedence
- Associate left to right

using statements

- Eliminate use of `std::` prefix
- Write `cout` instead of `std::cout`

```

1
2 // Using if statements, relational
3 // operators, and equality operators.
4 #include <iostream>
5
6 using std::cout; // program uses cout
7 using std::cin;  // program uses cin
8 using std::endl; // program uses endl
9
10 // function main begins program
11 int main()
12 {
13     int num1; // first number
14     int num2; // second number
15
16     cout << "Enter two integers,
17     << "the relationships they
18     cin >> num1 >> num2; //
19
20     if ( num1 == num2 )
21         cout << num1 << " is equal to " << num2 << endl;
22
23     if ( num1 != num2 )
24         cout << num1 << " is not equal to " << num2 << endl;
25

```

using statements eliminate need for **std::** prefix.

Declare variables.

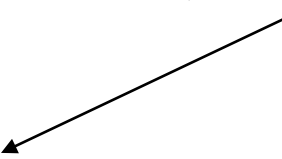
Can write **cout** and **cin** without **std::** prefix.

if structure compares values of **num1** and **num2** for equality. If condition is true (i.e., values are equal), execute this statement.

if structure compares values of **num1** and **num2** for inequality. If condition is true (i.e., values are not equal), execute this statement.

```
26  if ( num1 < num2 )
27      cout << num1 << " is less than " << num2 << endl;
28
29  if ( num1 > num2 )
30      cout << num1 << " is greater than " << num2 << endl;
31
32  if ( num1 <= num2 )
33      cout << num1 << " is less than or equal to "
34          << num2 << endl;
35
36  if ( num1 >= num2 )
37      cout << num1 << " is greater than or equal to "
38          << num2 << endl;
39
40  return 0; // indicate that program ended successfully
41
42  } // end function main
```

Statements may be split over several lines.



```
Enter two integers, and I will tell you
the relationships they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12
```

Basics of a C++ Program

switch

- Test variable for multiple values
- Series of **case** labels and optional **default** case

```
switch(variable) {  
    case value1:          // taken if variable == value1  
        statements  
        break;           // necessary to exit switch  
  
    case value2:  
  
    case value3:          // taken if variable == value2 or == value3  
        statements  
        break;  
  
    default:              // taken if none matches  
        statements  
        break;  
}
```

Basics of a C++ Program

Repetition Structures

while structure

- Makes loop continuation test at beginning
- Loop body may or may not execute depending on the condition

```
while ( condition )  
{  
    statement  
}
```

do/while structure

- Similar to **while** structure
- Makes loop continuation test at end, not beginning
- Loop body executes at least once

```
do {  
    statement  
} while ( condition );
```

for structure

- Makes loop continuation test at beginning
- Loop body may or may not execute depending on the condition

```
for(initialization; condition; update)  
{  
    statement  
}
```

Basics of a C++ Program

break statement

- Immediate exit from **while, for, do/while, switch**
- Program continues with first statement after structure

Common uses

- Escape early from a loop
- Skip the remainder of **switch**

continue statement

- Immediate jump to the start of **while, for, do/while**
- When a continue statement is encountered inside a loop, control jumps to the beginning of the loop for next iteration, skipping the execution of statements inside the body of loop for the current iteration

Basics of a C++ Program

Logical Operators

Used as conditions in loops, **if** statements

&& (logical **AND**)

- **true** if both conditions are **true**

```
if ( gender == 1 && age >= 65 )  
    ++seniorFemales;
```

|| (logical **OR**)

- **true** if either of condition is **true**

```
if ( semesterAverage >= 90 || finalExam >= 90 )  
    cout << "Student grade is A" << endl;
```

! (logical **NOT**, logical negation)

- Returns **true** when its condition is **false**, & vice versa

```
if ( !( grade == "F" ) )  
    cout << "The next grade is " << grade << endl;
```

```
if ( grade != "F" )  
    cout << "The next grade is " << grade << endl;
```

Structured-Programming Summary

All programs broken down into

- Sequence
- Selection
 - `if, if/else, or switch`
 - Any selection can be rewritten as an `if` statement
- Repetition
 - `while, do/while or for`
 - Any repetition structure can be rewritten as a `while` statement

ALGORITHMS

Computing problems

- Solved by executing a series of actions in a specific order

Algorithm a procedure determining

- Actions to be executed
- Order to be executed
- Example: recipe

Program control

- Specifies the order in which statements are executed

PSEUDOCODE

Pseudocode

- Artificial, informal language used to develop algorithms
- Similar to everyday English

Not executed on computers

- Used to think out program before coding
 - Easy to convert into C++ program
- **Only executable statements**
 - No need to declare variables

CONTROL STRUCTURES

Sequential execution

- Statements executed in order

Transfer of control

- Next statement executed *not* next one in sequence
- Structured programming – “goto”-less programming

3 control structures to build any program

- Sequence structure
 - Programs executed sequentially by default
- Selection structures
 - `if`, `if/else`, `switch`
- Repetition structures
 - `while`, `do/while`, `for`

CONTROL STRUCTURES

Flowchart

- Graphical representation of an algorithm
- Special-purpose symbols connected by arrows (flowlines)
- Rectangle symbol (action symbol)
 - Any type of action
- Oval symbol
 - Beginning or end of a program, or a section of code (circles)

Single-entry/single-exit control structures

- Connect exit point of one to entry point of the next
- Control structure stacking

IF SELECTION STRUCTURE

Selection structure

- Choose among alternative courses of action
- Pseudocode example:
 - If student's grade is greater than or equal to 60*
 - Print "Passed"*
- If the condition is **true**
 - Print statement executed, program continues to next statement
- If the condition is **false**
 - Print statement ignored, program continues
- Indenting makes programs easier to read
 - C++ ignores whitespace characters (tabs, spaces, etc.)

IF SELECTION STRUCTURE

Translation into C++

*If student's grade is greater than or equal to 60
Print "Passed"*

```
if ( grade >= 60 )  
    cout << "Passed";
```

Diamond symbol (decision symbol)

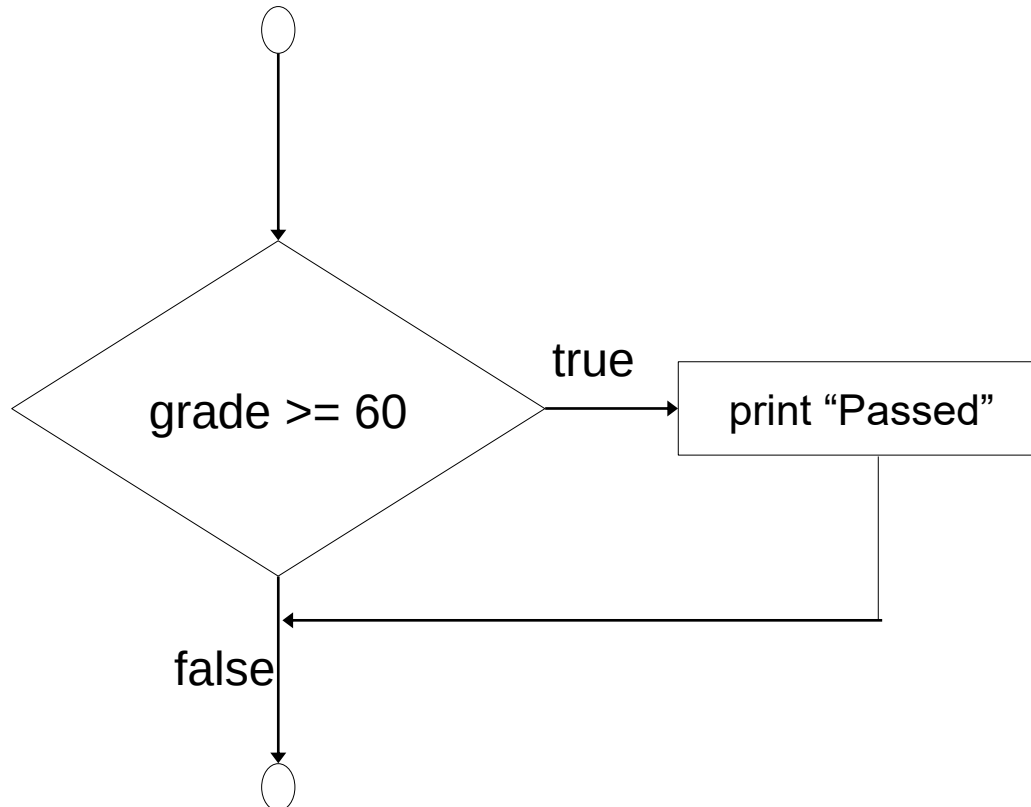
- Indicates decision is to be made
- Contains an expression that can be true or false
 - Test condition, follow path

if structure

- Single-entry/single-exit

IF SELECTION STRUCTURE

Flowchart of pseudocode statement



A decision can be made on any expression.

zero - **false**

nonzero - **true**

Example:

3 - 4 is **true**

IF/ELSE SELECTION STRUCTURE

if

- Performs action if condition true

if/else

- Different actions if conditions true or false

Pseudocode

```
if student's grade is greater than or equal to 60  
    print "Passed"  
else  
    print "Failed"
```

C++ code

```
if ( grade >= 60 )  
    cout << "Passed";  
else  
    cout << "Failed";
```

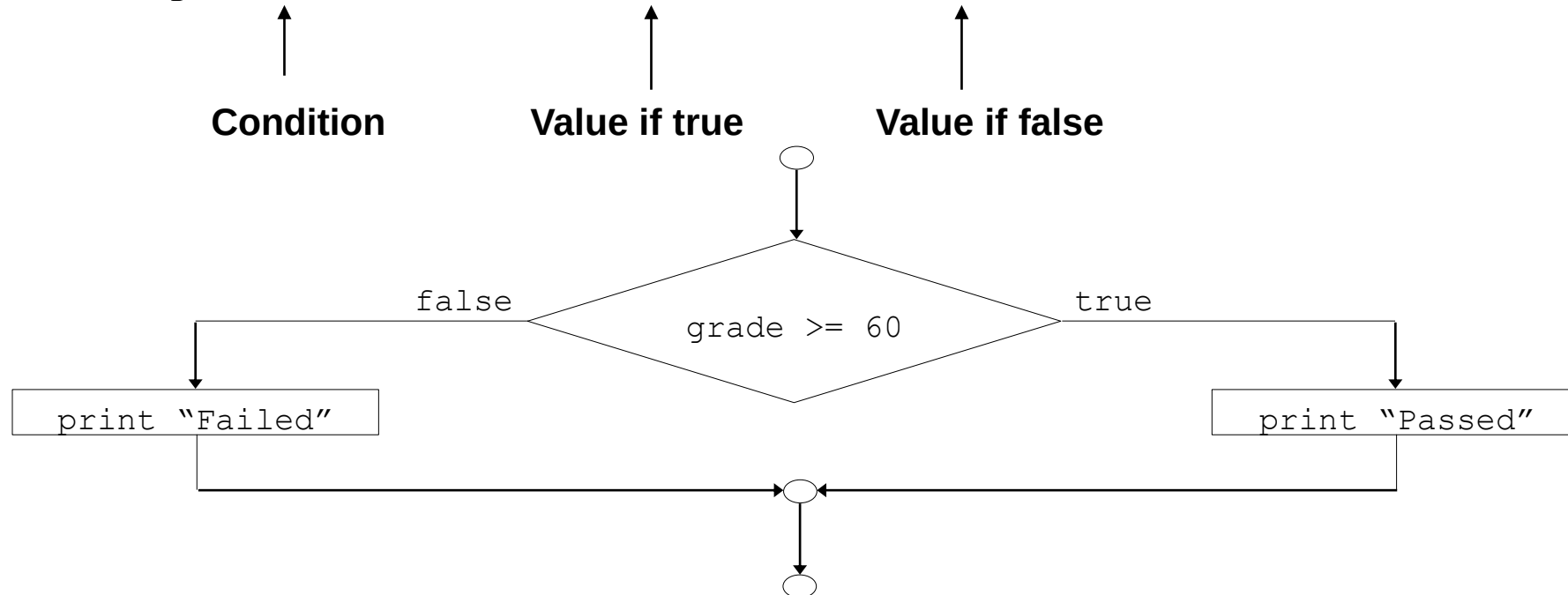
IF/ELSE SELECTION STRUCTURE

Ternary conditional operator (?:)

- Three arguments (condition, value if **true**, value if **false**)

Code could be written:

```
cout << ( grade >= 60 ? "Passed" : "Failed" );
```



IF/ELSE SELECTION STRUCTURE

Nested **if/else** structures

- One inside another, test for multiple cases
- Once condition met, other statements skipped

*if student's grade is greater than or equal to 90
Print "A"*

else

*if student's grade is greater than or equal to 80
Print "B"*

else

*if student's grade is greater than or equal to 70
Print "C"*

else

*if student's grade is greater than or equal to 60
Print "D"*

else

Print "F"

IF/ELSE SELECTION STRUCTURE

Example

```
if ( grade >= 90 )           // 90 and above
    cout << "A";
else if ( grade >= 80 )      // 80-89
    cout << "B";
else if ( grade >= 70 )      // 70-79
    cout << "C";
else if ( grade >= 60 )      // 60-69
    cout << "D";
else                          // less than 60
    cout << "F";
```

IF/ELSE SELECTION STRUCTURE

Compound statement

- Set of statements within a pair of braces

```
if ( grade >= 60 )  
    cout << "Passed.\n";  
else {  
    cout << "Failed.\n";  
    cout << "You must take this course again.\n";  
}
```

- Without braces,

```
cout << "You must take this course again.\n";  
always executed
```

Block

- Set of statements within braces

WHILE REPETITION STRUCTURE

Repetition structure

- Action repeated while some condition remains true
- Psuedocode

while there are more items on my shopping list

Purchase next item and cross it off my list

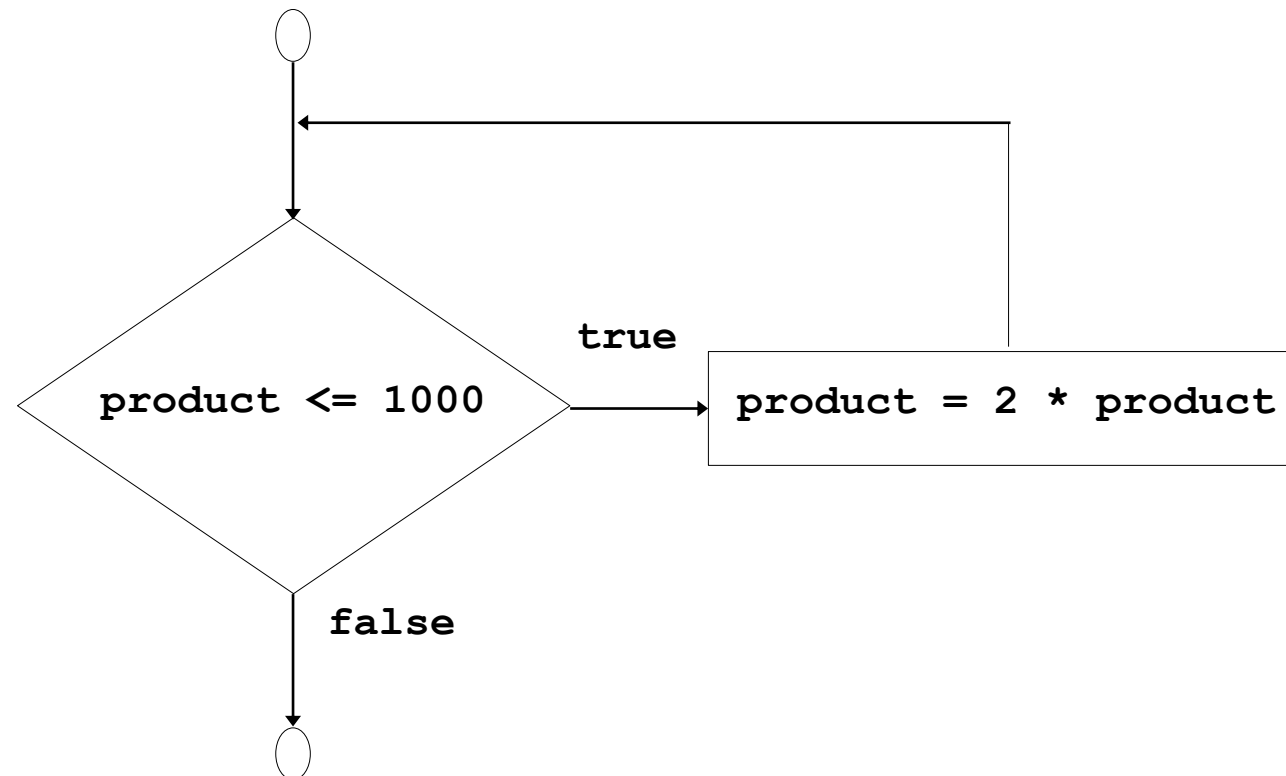
- **while** loop repeated until condition becomes false

Example

```
int product = 2;  
while ( product <= 1000 )  
    product = 2 * product;
```

WHILE REPETITION STRUCTURE

Flowchart of **while** loop



COUNTER-CONTROLLED REPETITION

Counter-controlled repetition

- Loop repeated until counter reaches certain value

Definite repetition

- Number of repetitions known

Example

A class of ten students took a quiz. The grades (integers in the range 0 to 100) for this quiz are available to you. Determine the class average on the quiz.

```
1  // Class average program with counter-controlled repetition.
2
3  #include <iostream>
4
5  using std::cout;
6  using std::cin;
7  using std::endl;
8
9  // function main begins program execution
10 int main()
11 {
12     int total;        // sum of grades input by user
13     int gradeCounter; // number of grade to be entered next
14     int grade;        // grade value
15     int average;      // average of grades
16
17     // initialization phase
18     total = 0;        // initialize total
19     gradeCounter = 1; // initialize loop counter
20
```

```
21 // processing phase
22 while ( gradeCounter <= 10 ) { // loop 10 times
23     cout << "Enter grade: "; // prompt for input
24     cin >> grade; // read grade from user
25     total = total + grade; // add grade to total
26     gradeCounter = gradeCounter + 1; // increment counter
27 }
28
29 // termination phase
30 average = total / 10;
31
32 // display result
33 cout << "Class average is " << average << endl;
34
35 return 0; // indicate program ended successfully
36
37 } // end function main
```

The counter gets incremented each time the loop executes. Eventually, the counter causes the loop to end.

```
Enter grade: 98
Enter grade: 76
Enter grade: 71
Enter grade: 87
Enter grade: 83
Enter grade: 90
Enter grade: 57
Enter grade: 79
Enter grade: 82
Enter grade: 94
Class average is 81
```

SENTINEL-CONTROLLED REPETITION

Suppose problem becomes:

Develop a class-averaging program that will process an arbitrary number of grades each time the program is run

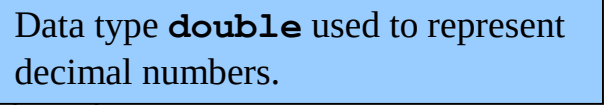
- Unknown number of students
- How will program know when to end?

Sentinel value

- Indicates “end of data entry”
- Loop ends when sentinel input
- Sentinel chosen so it cannot be confused with regular input
 - -1 in this case

```
1
2 // Class average program with sentinel-controlled repetition.
3 #include <iostream>
4
5 using std::cout;
6 using std::cin;
7 using std::endl;
8 using std::fixed;
9
10 #include <iomanip> // parameterized stream manipulators
11
12 using std::setprecision; // sets numeric output precision
13
14 // function main begins program execution
15 int main()
16 {
17     int total; // sum of grades
18     int gradeCounter; // number of grades entered
19     int grade; // grade value
20
21     double average; // number with decimal point for average
22
23     // initialization phase
24     total = 0; // initialize total
25     gradeCounter = 0; // initialize loop counter
```

Data type **double** used to represent decimal numbers.



```

26
27 // processing phase
28 // get first grade from user
29 cout << "Enter grade, -1 to end: "; // prompt for input
30 cin >> grade; // read grade from user
31
32 // loop until sentinel value read from user
33 while ( grade != -1 ) {
34     total = total + grade; // add grade to total
35     gradeCounter = gradeCounter + 1; // increment counter
36
37     cout << "Enter grade, -1 to end: ";
38     cin >> grade;
39
40 } // end while
41
42 // termination phase
43 // if user entered at least one grade
44 if ( gradeCounter != 0 ) {
45
46     // calculate average of all grades entered
47     average = static_cast< double >( total ) / gradeCounter;
48

```

`static_cast<double>()` treats `total` as a `double` temporarily (casting).

Required because dividing two integers truncates the remainder.

`gradeCounter` is an `int`, but it gets *promoted* to `double`.

```

49 // display average with two digits of precision
50 cout << "Class average is " << setprecision( 2 )
51     << fixed << average << endl;
52
53 } // end if part of if/else
54
55 else // if no grades were entered, output appropriate message
56     cout << "No grades were entered" << endl;
57
58 return 0; // indicate program ended
59
60 } // end function main

```

setprecision(2) prints two digits past decimal point (rounded to fit precision).

Programs that use this must include **<iomanip>**

fixed forces output to print in fixed point format (not scientific notation). Also, forces trailing zeros and decimal point to print.

Include **<iostream>**

```

Enter grade, -1 to end: 75
Enter grade, -1 to end: 94
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 70
Enter grade, -1 to end: 64
Enter grade, -1 to end: 83
Enter grade, -1 to end: 89
Enter grade, -1 to end: -1
Class average is 82.50

```

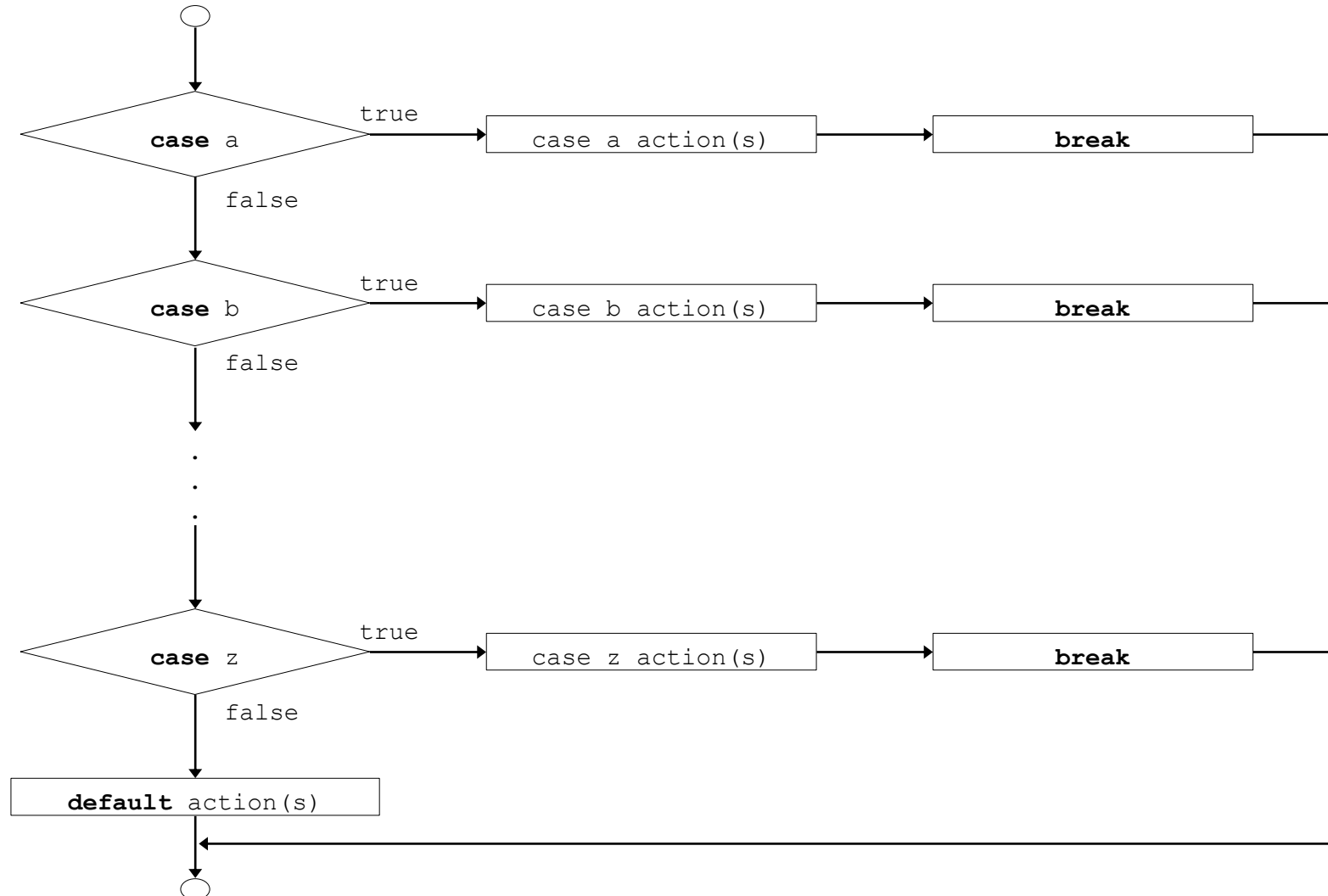
SWITCH MULTIPLE-SELECTION STRUCTURE

switch

- Test variable for multiple values
- Series of **case** labels and optional **default** case

```
switch ( variable ) {  
    case value1:          // taken if variable == value1  
        statements  
        break;           // necessary to exit switch  
  
    case value2:  
    case value3:          // taken if variable == value2 or == value3  
        statements  
        break;  
  
    default:              // taken if none matches  
        statements  
        break;  
}
```

SWITCH MULTIPLE-SELECTION STRUCTURE



SWITCH MULTIPLE-SELECTION STRUCTURE

Example upcoming

- Program to read grades (A-F)
- Display number of each grade entered

Details about characters

- Single characters typically stored in a **char** data type
 - **char** a 1-byte integer, so **chars** can be stored as **ints**
- Can treat character as **int** or **char**
 - 97 is the numerical representation of lowercase 'a' (ASCII)
 - Use *single quotes* to get numerical representation of character

```
cout << "The character (" << 'a' << ") has the value "  
      << static_cast< int > ( 'a' ) << endl;
```

Prints

The character (a) has the value 97

```
1  // Fig. 2.22: fig02_22.cpp
2  // Counting letter grades.
3  #include <iostream>
4
5  using std::cout;
6  using std::cin;
7  using std::endl;
8
9  // function main begins program execution
10 int main()
11 {
12     int grade;    // one grade
13     int aCount = 0; // number of As
14     int bCount = 0; // number of Bs
15     int cCount = 0; // number of Cs
16     int dCount = 0; // number of Ds
17     int fCount = 0; // number of Fs
18
19     cout << "Enter the letter grades." << endl
20          << "Enter the EOF character to end input." << endl;
21
```

```

22 // loop until user types end-of-file key sequ
23 while ( ( grade = cin.get() ) != EOF ) {
24
25 // determine which grade was input
26 switch ( grade ) { // switch structure nested in while
27
28 case 'A': // grade was uppercase A
29 case 'a': // or lowercase a
30 ++aCount; // i
31 break; // nee
32
33 case 'B': // gra
34 case 'b': // or l
35 ++bCount; // i
36 break; // exi
37
38 // gra
39 // or lowercase c
40 ++cCount; // increment cCount
41 break; // exit switch
42

```

break causes **switch** to end and the program continues with the first statement after the **switch** structure.

cin.get() uses dot notation . This function gets 1 character from the keyboard (after *Enter* pressed), and it is assigned to **grade**.

cin.get() returns EOF (end-of-file) after the EOF character is input, to indicate the end of data. EOF may be ctrl-d or ctrl-z, depending on your OS.

Compares **grade** (an **int**) to the numerical representations of **A** and **a**.

Assignment statements have a value, which is the same as the variable on the left of the **=**. The value of this statement is the same as the value returned by **cin.get()**.

This can also be used to initialize multiple variables:
a = b = c = 0;

```

43     case 'D':        // grade was uppercase D
44     case 'd':        // or lowercase d
45         ++dCount;    // increment dCount
46         break;       // exit switch
47
48     case 'F':        // grade was uppercase F
49     case 'f':        // or lowercase f
50         ++fCount;    // increment fCount
51         break;       // exit switch
52
53     case '\n':       // ignore newlines,
54     case '\t':       // tabs,
55     case ' ':        // and spaces in input
56         break;       // exit switch
57
58     default:         // catch all other characters
59         cout << "Incorrect letter grade entered."
60             << " Enter a new grade." << endl;
61         break;       // optional; will exit switch anyway
62
63 } // end switch
64
65 } // end while
66

```

This test is necessary because *Enter* is pressed after each letter grade is input. This adds a newline character that must be removed. Likewise, we want to ignore any whitespace.

Notice the **default** statement, which catches all other cases.

```
67 // output summary of results
68 cout << "\n\nTotals for each letter grade are:"
69     << "\nA: " << aCount // display number of A grades
70     << "\nB: " << bCount // display number of B grades
71     << "\nC: " << cCount // display number of C grades
72     << "\nD: " << dCount // display number of D grades
73     << "\nF: " << fCount // display number of F grades
74     << endl;
75
76 return 0; // indicate successful termination
77
78 }
```

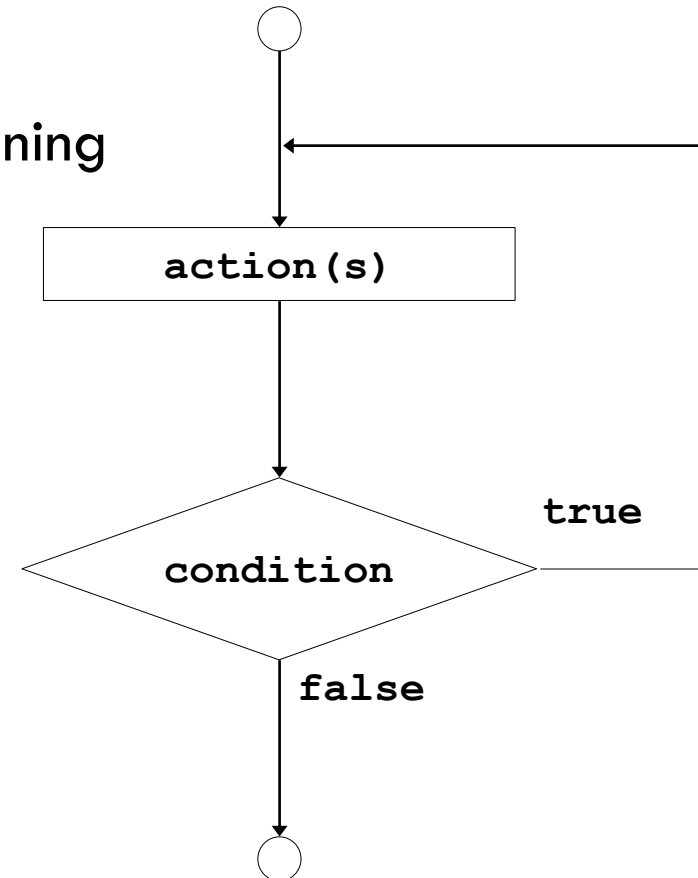
DO/WHILE REPETITION STRUCTURE

Similar to **while** structure

- Makes loop continuation test at end, not beginning
- Loop body executes at least once

Format

```
do {  
    statement  
} while ( condition );
```



```

1  // Fig. 2.24: fig02_24.cpp
2  // Using the do/while repetition structure.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  // function main begins program execution
9  int main()
10 {
11     int counter = 1;           // initialize
12
13     do {
14         cout << counter << " "; // display counter
15     } while ( ++counter <= 10 ); // end do/while
16
17     cout << endl;
18
19     return 0; // indicate successful termination
20
21 } // end function main

```

Notice the preincrement in loop-continuation test.

BREAK AND CONTINUE STATEMENTS

`break` statement

- Immediate exit from `while, for, do/while, switch`
- Program continues with first statement after structure

Common uses

- Escape early from a loop
- Skip the remainder of `switch`

```

1  // Fig. 2.26: fig02_26.cpp
2  // Using the break statement in a for structure.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  // function main begins program execution
9  int main()
10 {
11
12     int x; // x declared here so it can be used after the loop
13
14     // loop 10 times
15     for ( x = 1; x <= 10; x++ ) {
16
17         // if x is 5, terminate loop
18         if ( x == 5 )
19             break; // break loop only if x is 5
20
21         cout << x << " "; // display value of x
22
23     } // end for
24
25     cout << "\nBroke out of loop when x became " << x << endl;

```

Exits **for** structure when **break** executed.

LOGICAL OPERATORS

Used as conditions in loops, **if** statements

&& (logical **AND**)

- **true** if both conditions are **true**

```
if ( gender == 1 && age >= 65 )  
    ++seniorFemales;
```

|| (logical **OR**)

- **true** if either of condition is **true**

```
if ( semesterAverage >= 90 || finalExam >= 90 )  
    cout << "Student grade is A" << endl;
```

LOGICAL OPERATORS

! (logical **NOT**, logical negation)

- Returns **true** when its condition is **false**, & vice versa

```
if ( !( grade == sentinelValue ) )  
    cout << "The next grade is " << grade << endl;
```

Alternative:

```
if ( grade != sentinelValue )  
    cout << "The next grade is " << grade << endl;
```

CONFUSING EQUALITY (==) AND ASSIGNMENT (=) OPERATORS

Common error

- Does not typically cause syntax errors

Aspects of problem

- Expressions that have a value can be used for decision
 - Zero = false, nonzero = true
- Assignment statements produce a value (the value to be assigned)

CONFUSING EQUALITY (==) AND ASSIGNMENT (=) OPERATORS

Example

```
if ( payCode == 4 )  
    cout << "You get a bonus!" << endl;
```

- If paycode is 4, bonus given

If == was replaced with =

```
if ( payCode = 4 )  
    cout << "You get a bonus!" << endl;
```

- Paycode set to 4 (no matter what it was before)
- Statement is true (since 4 is non-zero)
- Bonus given in every case

CONFUSING EQUALITY (==) AND ASSIGNMENT (=) OPERATORS

Lvalues

- Expressions that can appear on left side of equation
- Can be changed (i.e., variables)

`x = 4;`

Rvalues

- Only appear on right side of equation
- Constants, such as numbers (i.e. cannot write `4 = x;`)

Lvalues can be used as rvalues, but not vice versa

STRUCTURED-PROGRAMMING SUMMARY

Structured programming

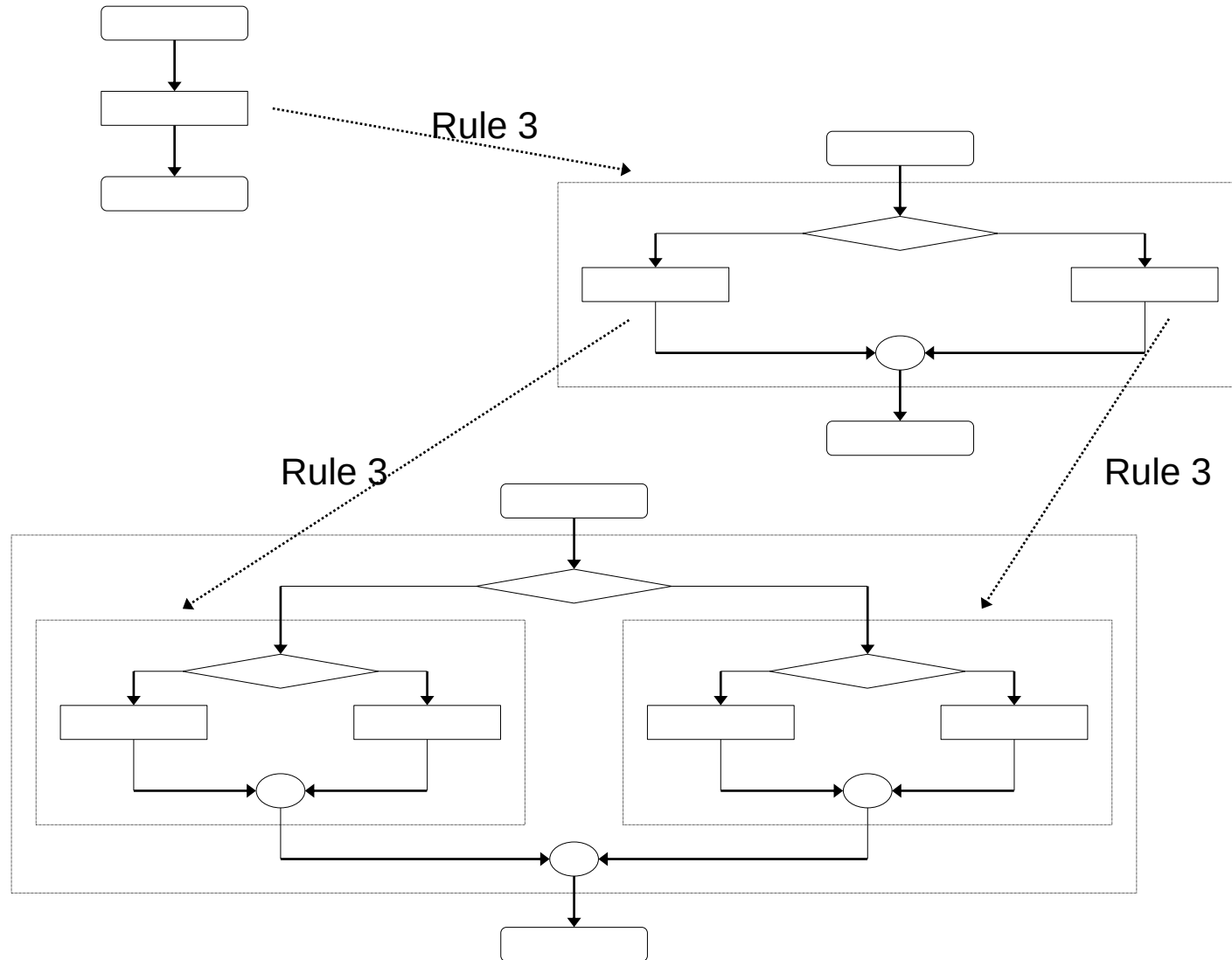
- Programs easier to understand, test, debug and modify

Rules for structured programming

- Only use single-entry/single-exit control structures
- Rules
 - 1) Begin with the “simplest flowchart”
 - 2) Any rectangle (action) can be replaced by two rectangles (actions) in sequence
 - 3) Any rectangle (action) can be replaced by any control structure (sequence, if, if/else, switch, while, do/while or for)
 - 4) Rules 2 and 3 can be applied in any order and multiple times

STRUCTURED-PROGRAMMING SUMMARY

Representation of Rule 3 (replacing any rectangle with a control structure)



STRUCTURED-PROGRAMMING SUMMARY

All programs broken down into

- Sequence
- Selection
 - `if`, `if/else`, or `switch`
 - Any selection can be rewritten as an `if` statement
- Repetition
 - `while`, `do/while` or `for`
 - Any repetition structure can be rewritten as a `while` statement